Diseño de solución Prueba Técnica Pinflag

Datos Personales

- Candidato: **Diego Antonio Guerra Joselin**
- Posición aspirante: **DevOps**

1. Objetivos a resolver

- **Trazabilidad y observabilidad**: Se cumple si registra en base de datos la regla ganadora y las colisiones descartadas por cada pedido para auditoría.
- **Completitud del pedido**: Se cumple si una regla por defecto (fallback) opera para que ningún pedido quede sin courier asignado.
- **Control de excepciones**: Manejar errores controlados ante falta de cobertura, desempates idénticos o intentos de manipulación de datos.
- **Verificación en creación de reglas**: Validar en backend rangos coherentes (min/max), evitar prioridades duplicadas y alertar sobre solapamientos.
- **Simulación previa**: Disponer de un entorno de prueba en tiempo real para evaluar pedidos sin escribir en la tabla de pedidos.

2. Diseño de la solución

Para representar reglas como tal, se genera una combinación en base a las acciones que aplican a cada una, optando por un enfoque más pragmático apuntando a que el pedido se cumpla y llegue a destino del consumidor. De esta forma una **regla** que creé contendrá las siguientes características basadas en la entrada de datos (incluiré campos principales, pero en BD estará más detallado):

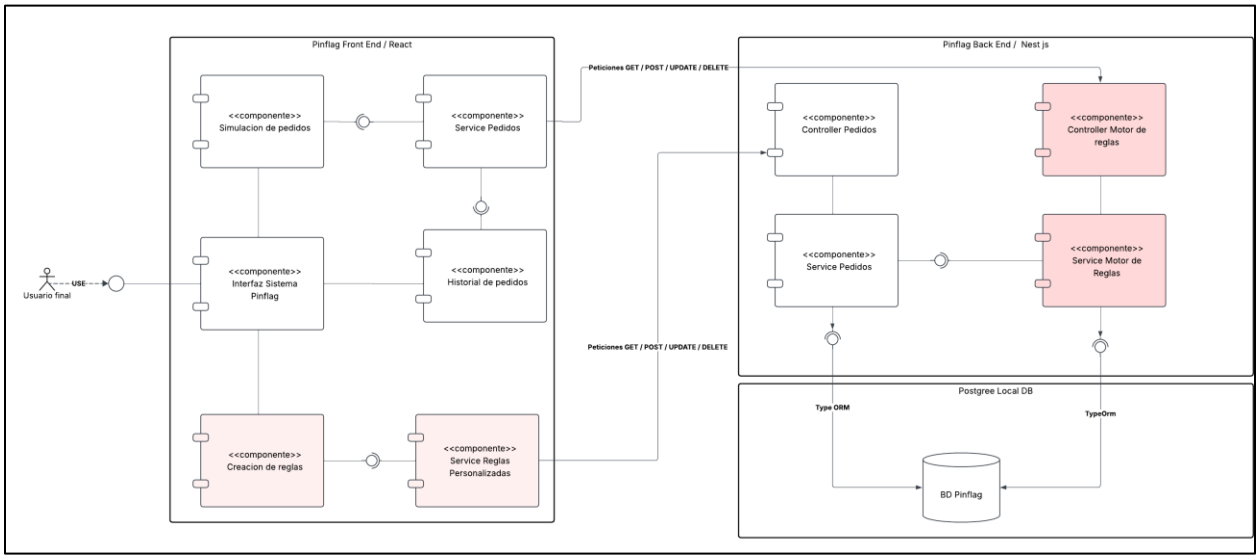
id	Prioridad	Valor min	Valor max	Peso	Cobertura	Courier	Tipos de entrega	Despacho Gratis	Costo envió
1	3	60000	null	20	100	Blue Express	Despacho a domicilio, Retiro en tienda, Retiro en punto	True	0
2	0	0	null	null	null	FedEx	Despacho a domicilio	False	4500

Las **acciones** están envueltas en el sistema de reglas que se aplicará y guardará en la base de datos, por ejemplo, como se observa en la tabla, en base a su prioridad se establecerá cual predomina sobre otras ejecutando acciones como **asignación de courier**, **agregar o quitar tipos de entrega** y **modificar precios de envío**.

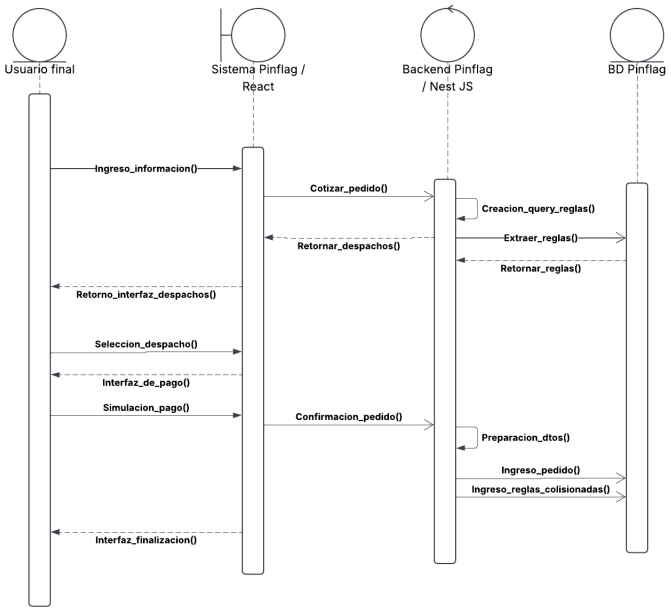
Debido a que existirán reglas que por prioridad y condiciones colisionarán, éstas se registrarán en una tabla de logs en la base de datos que estará ligado al **ID del pedido** con dichas reglas que colisionaron para facilitar su seguimiento.

El diseño de la solución operara de la siguiente forma:

1. **Diagrama de componentes (Front End y Back End)**



2. **Diagrama de secuencia (enfoque en simulación de pedido)**



El diagrama de componentes visualizado comprende la comunicación y acción de los mismos referentes a las operaciones que ejecutará el sistema. Los componentes marcados en rojo indican dónde vivirá las lógicas más importantes referente a las reglas y sobre las cuales se ejecutarán acciones como:

- Contabilizar total de reglas creadas.
- Detectar colisiones de reglas en la creación.
- Guardar reglas colisionadas en base de datos como log table para abordar inconsistencias.
- Ejecutar sistema de ranking de reglas según la prioridad del cliente.

El principal componente involucrado será gestionado por el motor de reglas del backend que guardará logs, administrará la lógica de aciertos y retornará información en cuanto a registros.

El diagrama de secuencia aborda el principal proceso de simulación de un pedido con el motor de reglas operando como tal. De esta forma se puede visualizar cómo operará el sistema con sus componentes y servicios respectivos.

Riesgos y características no resueltas por el modelo.

Comenzando por el diseño de la solución y qué es lo que no resuelve: el conflicto de reglas que tengan variaciones que pueden reflejarse de manera idéntica. Por ejemplo, si reglas con rangos de precios comparten valores entre 0 y 40000, incluyendo la prioridad idéntica 2 y tomando en cuenta que uno de ellos tiene envió gratis y el otro no, la asignación de courier (acción) puede resultar incorrecta.

En base a esta situación identificada se detallan dos riesgos y mitigaciones son las siguientes:

1. **Riesgo: Empate de colisiones en ranking.**
    a. **Mitigación:** En la entidad de reglas se debe adecuar algún identificador secundario para poder generar un desempate y elegir una regla correspondiente.
2. **Riesgo: Asignación de prioridades en reglas de acuerdo con necesidades del cliente.**
    a. **Mitigación:** Solución de tipo drag and drop que establezca prioridades ordenadas según las necesidades del cliente.

Uso de IA.

En qué partes use IA: Ideación arquitectónica del motor de reglas, contraste entre modelos (DMN/Decisión Tables vs. matrices cerradas) y estructuración de diagramas UML (verificación de objetos).

Prototipado front end con React, modelamiento de entidades para Nest JS y validación de transacciones y registro en base de datos.

Sugerencias entregadas: Sugerencias iniciales de bibliotecas (como json-rules-engine), modelos desacoplados de evaluación, interfaces.

Modificaciones y contenido descartado: Resolución al usuario final descartando selectores/modales en frontend, modificado interfaces y propiedades extensas para entidades, modificación DMN hacia una arquitectura de matriz de reglas pragmáticas con regla fallback para evitar pedidos huérfanos.